

Zero to Hero Study Handbook: NL2SQL Agent

How to use this handbook:

1. Read modules in order. Each module builds on the previous one.
2. Keep the code open while reading. Every concept here maps to real files/functions in this repo.
3. After Module 5, use the checklist to validate end-to-end understanding.

Module 1: Foundations & Architecture

1.1 What this project does

`nl2sql-agent` is a local-first natural-language-to-SQL system. A user asks a plain-English question (CLI or Streamlit UI), the agent generates SQL, validates it for safety, runs it on read-only SQLite or operator-configured PostgreSQL, and returns a natural-language answer plus SQL/result context.

Primary use cases in this repo:

1. Local analytics assistant over SQLite (`company.db` by default) or a restricted PostgreSQL schema.
2. Safe text-to-SQL experimentation with policy controls (joins, subqueries, aggregates, CTE).
3. Multi-provider runtime where Ollama is default and Hugging Face, OpenAI, Anthropic, Gemini, and xAI are optional.
4. Approval-first analysis of session-scoped uploaded SQLite databases.
5. Result-based model evaluation across accuracy, safety, latency, retries, and token usage.

1.2 Core paradigms and patterns used

Definitions first, then where they appear in this codebase:

Modular architecture: Definition: split the system into small modules with one clear job each. In this repo: `config`, `db`, `security`, `llm`, `prompts`, `agent`, `ui`, `utils` under `src/nl2sql_agent/`.

Factory pattern: Definition: one function returns different implementations behind a shared interface. In this repo: `build_chat_model()` returns provider-specific chat models in `src/nl2sql_agent/llm/factory.py`.

Stateful workflow (graph/state-machine orchestration): Definition: execution is modeled as nodes and conditional transitions over shared state. In this repo: `NL2SQLAgent.get_workflow()` in `src/nl2sql_agent/agent/workflow.py` using LangGraph.

Policy-based security (allow-list): Definition: explicitly allow known-safe behavior instead of only blocking known-bad patterns. In this repo: `SQLPolicy` + `validate_sql()` in `src/nl2sql_agent/security/sql_validator.py`.

Typed data contracts: Definition: key data objects have explicit types and field names. In this repo: `AgentState` (`agent/state.py`) and `QueryResult` (`db/database.py`).

Settings singleton with environment binding: Definition: one cached settings object is reused across the app and can be reset in tests. In this repo: `Settings`, `get_settings()`, `reset_settings_cache()` in `config/settings.py`.

Defense in depth: Definition: multiple independent controls prevent one failure from becoming data loss. In this repo: AST validation, table authorization, preflight compilation, read-only SQLite, progress limits, and redacted audits.

1.3 Architecture description

Key components and interactions:

Entrypoints: - CLI: `src/nl2sql_agent/cli.py (main(), subcommands ask/config/serve/eval)` - UI: `src/nl2sql_agent/ui/streamlit_app.py (main())`

Model construction: - `build_chat_model()` selects provider and returns a `BaseChatModel`.

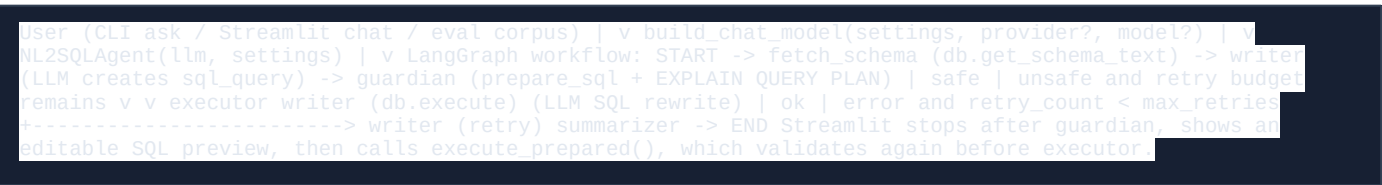
Core agent workflow: - `NL2SQLAgent` builds a `LangGraph` with nodes: `fetch_schema` -> `writer` -> `guardian` -> `executor` -> `summarizer`

Data access: - `Database.ensure_schema()` creates/seeds tables. - `Database.execute()` runs SQL on a hardened read-only connection and returns `QueryResult`.

Security gate: - `prepare_sql()` blocks non-SELECT, multi-statement, dangerous functions, disallowed tables, and policy violations, then enforces the row limit.

Prompting: - SQL-writing and summarization prompts in `prompts/templates.py`.

1.4 Main flow diagram (ASCII)



Schema fetching keeps one read-only connection open, scores every table and column identifier once, and loads foreign-key metadata only for the tables that will be sent to the writer.

Module 2: Repository Map

Files a new contributor should learn first (in practical reading order):

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
Launch NL2SQL Agent.cmd	Windows double-click startup	checks uv, then runs <code>nl2sql-agent serve</code>	locked environment, loopback port 8501, visible logs
pyproject.toml	Packaging, dependencies, scripts, lint/type/test config	<code>[project.scripts]</code> <code>nl2sql-agent</code>	<code>requires-python</code> , direct dependencies, transitive constraints, Ruff/ty/Pytest settings
src/nl2sql_agent/cli.py	CLI entrypoint for ask/config/serve/eval	<code>main</code> , <code>cmd_ask</code> , <code>cmd_eval</code> , <code>cmd_config</code> , <code>cmd_serve</code>	Provider/model overrides, eval threshold, report and cost options
src/nl2sql_agent/ui/streamlit_app.py	Approval-first multipage Streamlit orchestration	<code>main</code> , <code>_resolve_database</code> , <code>_build_agent</code>	Chat context, temporary upload workspace, pending SQL, active database context
src/nl2sql_agent/ui/components.py	UI widgets and result rendering	<code>render_sidebar</code> , <code>render_chat_history</code> , <code>render_run_result</code>	Database source, provider/model, results, traces, plans, metrics, CSV download
src/nl2sql_agent/ui/pages.py	Costs, Sessions, Insights, and Pricing views	<code>render_costs_page</code> , <code>render_sessions_page</code> , <code>render_insights_page</code> , <code>render_pricing_page</code>	Dashboards, saved-session management, rule editing, budgets

src/ nl2sql_agent/ persistence.py	Versioned local state database	StateStore	Allowlisted messages, approved SQL, usage, price snapshots, plans, metrics; never raw results or secrets
src/ nl2sql_agent/ ui/ database_upload.py	Untrusted upload validation/storage	validate_sqlite_upload, save_sqlite_upload	Extension, size, SQLite header, content digest
src/ nl2sql_agent/ agent/ workflow.py	Full and two-phase workflow routing	NL2SQLAgent, prepare, stream_prepare, execute_prepared, run, stream	Run ID, allowlist, SQL safety, retries, traces, token usage
src/ nl2sql_agent/ agent/ state.py	Typed workflow contract	AgentState	Fields for schema, SQL, retries, raw rows, columns, row_count
src/ nl2sql_agent/ security/ sql_validator.py	SQL AST validation and executable preparation	SQLPolicy, validate_sql, prepare_sql, referenced_tables	Feature toggles, structural limits, table allowlist, enforced LIMIT
src/ nl2sql_agent/ db/ database.py	Read-only SQLite schema/query layer	Database, QueryResult, list_tables, get_schema_text, preflight, execute	Timeout, VM steps, row limit, normalized plans/metrics, schema ranking
src/ nl2sql_agent/ db/ postgres.py	Strict read-only PostgreSQL layer	PostgresDatabase	Restricted role verification, one-schema introspection, timeouts, JSON plans
src/ nl2sql_agent/ evaluation/ runner.py	Result and safety evaluation	EvalCase, EvaluationRunner, EvaluationReport	Reference result comparison, threshold metrics, integrity digest
src/ nl2sql_agent/ utils/ audit.py	Privacy-preserving JSONL audit	AuditLogger, redact_sql, hash_text	Hashed questions, literal-redacted SQL
src/ nl2sql_agent/ db/ seed.py	Demo seed data	SEED_DEPARTMENTS, SEED_EMPLOYEES	Department/employee tuples loaded via INSERT OR IGNORE
src/ nl2sql_agent/ llm/ factory.py	Provider-specific model building and model listing	build_chat_model, list_models, fallback_models, _build_*, _list_ollama, LLMPProviderError	Six providers; medium hosted reasoning; Ollama-only live discovery
src/ nl2sql_agent/ llm/ pricing.py	Per-call hosted-model pricing	PricingRule, UsageRecord, calculate_cost	Effective dates, cache/batch/fast rates, long-context thresholds, exact decimal arithmetic

<code>src/ nl2sql_agent/ config/ settings.py</code>	Central runtime config and model policy	Settings, <code>default_model_for</code> , <code>supported_models_for</code> , <code>validate_model_for</code> , <code>env_var_for</code>	<code>env_prefix="NL2SQL_"</code> , <code>provider/model</code> allow-lists, credentials, DB/SQL/log fields
<code>src/ nl2sql_agent/ prompts/ templates.py</code>	Prompt templates and formatting helpers	<code>SQL_WRITER_SYSTEM</code> , <code>SQL_WRITER_USER</code> , <code>SUMMARIZER_SYSTEM</code> , <code>SUMMARIZER_USER</code> , <code>error_section</code> , <code>format_data</code>	Prompt placeholders <code>{schema}</code> , <code>{question}</code> , <code>{error_section}</code> , <code>{sql}</code> , <code>{data}</code> , <code>{error}</code>
<code>src/ nl2sql_agent/ utils/ text.py</code>	SQL/text normalization utilities	<code>strip_sql_fences</code> , <code>truncate</code>	Regex constants <code>_SQL_FENCE_RE</code> , <code>_LEADING_SQL_TOKEN_RE</code>
<code>src/ nl2sql_agent/ utils/ logging.py</code>	Unified Loguru logging setup	<code>configure_logging</code> , <code>get_logger</code>	<code>level</code> , <code>json</code> , <code>stdlib</code> interception for <code>httpx/httpcore/...</code>
<code>tests/ unit/ test_agent.py</code>	Workflow behavior contract	Routing/retry/fallback tests for node methods and <code>run/stream</code>	Asserts on <code>retry_count</code> , <code>error</code> , <code>final_answer</code> , node names
<code>tests/ unit/ test_sql_validator.py</code>	Security policy contract	Safety/forbidden/function/subquery/join/CTE/function tests	Configuration allow-list behavior and failure cases

Module 3: Core Execution Flows

This module explains the real operational paths using concrete symbols and data shapes.

3.1 Core data contracts

AgentState shape (`src/nl2sql_agent/agent/state.py`)

```
# TypedDict, total=False { "run_id": str, "question": str, "schema": str, "allowed_tables": list[str],  
"sql_query": str, "sql_safe": bool, "sql_unsafe_reason": str, "result": str, "raw_rows": list[tuple[object,  
...]], "columns": list[str], "row_count": int, "csv_data": str, "truncated": bool, "error": str,  
"retry_count": int, "max_retries": int, "trace": list[dict[str, object]], "token_usage": dict[str, int],  
"final_answer": str, }
```

Important note: **AgentState** is declared with `total=False`. That means each node can return partial updates instead of a full state object.

QueryResult shape (`src/nl2sql_agent/db/database.py`)

```
QueryResult( columns: tuple[str, ...], rows: tuple[tuple[object, ...], ...], row_count: int, truncated:  
bool )
```

Sidebar config payload (`src/nl2sql_agent/ui/components.py`)

```
{ "data_source": str, # "Demo" | "Upload" "provider": Provider, # ollama | huggingface | openai | anthropic  
| gemini | xai "model": str | None, "api key": str | None, }
```

Initial workflow input shape (`run/stream` in `src/nl2sql_agent/agent/workflow.py`)

Both `run()` and `stream()` start with:

```
{ "run_id": str(uuid4()), "question": question, "retry_count": 0, "max_retries": int(max_retries or
self.settings.max_retries), "error": "", "trace": [], "token_usage": {}, "allowed_tables":
sorted(self.allowed_tables), }
```

3.2 Flow A: CLI ask request

Entrypoint chain:

1. `main()` parses args via `build_parser()`.
2. `cmd_ask(args)` calls `_build_agent(...)`.
3. `_build_agent`: - loads `settings = get_settings()` - applies CLI overrides (`provider/model/api_key`) - builds `llm = build_chat_model(settings)` - returns `NL2SQLAgent(llm, settings=settings)`
4. `cmd_ask` calls `agent.run(args.question)`.
5. Prints `result["final_answer"]`; if `--show-sql`, also prints `result["sql_query"]`.

Short code fragment from real path:

```
result = agent.run(args.question) print(result.get("final_answer", "")) if args.show_sql: print("\n--- SQL
---\n", result.get("sql_query", ""), sep="")
```

Input shape: - `args.question`: `str` - optional overrides: `args.provider`, `args.model`, `args.api_key`

Output shape: - terminal text; underlying `result` dict includes `final_answer`, `sql_query`, `error`, `columns`, `raw_rows`, `row_count`.

3.3 Flow B: Streamlit chat request

Entrypoint chain:

1. `streamlit_app.main()` configures logging, local state, and five native pages.
2. `render_sidebar()` selects Demo/Upload/PostgreSQL and the model provider.
3. Uploads are validated, stored under a content digest, and opened read-only.
4. The user selects allowed tables and whether uploaded sample rows may enter prompts.
5. `agent.stream_prepare(user_query)` emits schema, writer, and guardian updates.
6. Safe SQL is stored as `pending_query` and shown in an editable text area.
7. Run calls `agent.execute_prepared(...)`, which validates the edited SQL again.
8. Current result rows and CSV remain in memory. The state store persists only messages, approved SQL, context, usage/cost snapshots, and bounded metrics.
9. `render_run_result()` prices each provider-reported model call with the effective local rule and shows plan/runtime warnings.

Short code fragment from real path:

```
for node_name, update in agent.stream_prepare(user_query): final_state.update(update) result =
agent.execute_prepared(final_state, sql_query=edited_sql)
```

Input shape: - `user_query`: `str` - sidebar config from `render_sidebar()`

Output shape: - Editable SQL preview, answer, dataframe, CSV, stage timings, token counts, normalized plan/metrics, warnings, and an estimated hosted-model cost.

Each model call records provider-reported input/output/cache usage. The effective local rule applies its actual request mode and long-context threshold, then freezes a price snapshot with the approved run. Estimates remain local guidance rather than provider invoices.

3.4 Flow C: Agent internals (LangGraph node-by-node)

`NL2SQLAgent.get_workflow()` defines this graph:

`fetch_schema(state)`: - ranks allowed tables against the question and includes FK neighbors - optionally includes bounded sample rows - returns schema, allowlist, and a timing trace

`write_sql(state)`: - reads: `question`, `schema`, `previous error/sql_unsafe_reason` - builds `SQL_WRITER_USER` prompt - LLM call with system+user messages - cleans output via `strip_sql_fences()` - returns:

- `sql_query`
- incremented `retry_count`
- clears stale `error`, `sql_unsafe_reason`, `result`

`check_security(state)`: - runs `prepare_sql(sql, policy, allowed_tables)` - canonicalizes and enforces the executable LIMIT - runs `Database.preflight()` using `EXPLAIN QUERY PLAN` - records a redacted `prepared` or `blocked` audit event

`route_after_security(state)`: - if safe -> `"executor"` - if blocked and retry budget remains -> `"writer"` - otherwise -> `"summarizer"`

`execute_sql(state)`: - calls `self.db.execute(sql)` - on success returns markdown, rows, columns, count, CSV, truncation state, and timing - on `sqlite3.Error` returns error string and empty data containers

`route_after_execute(state)`: - if error and `retry_count < max_retries` -> `"writer"` - otherwise -> `"summarizer"`

`summarize_result(state)`: - builds summarizer prompt from `question`, `sql_query`, formatted `result`, `error` - LLM generates `final_answer` - fallback path via `_fallback_answer` if LLM fails or returns empty - returns `{"final_answer": answer, "result": answer}`

`prepare()/stream_prepare()` use a smaller graph that ends after guardian. `execute_prepared()` re-enters guardian before executor. `run()` and `stream()` continue to use the end-to-end graph for automation compatibility.

3.5 Flow D: Database initialization + seed behavior

Where it happens:

1. For a managed demo in `NL2SQLAgent.__init__`: - `self.db = Database(...)` - `self.db.ensure_schema(seed=self.settings.db_seed)`

Injected databases are never initialized or seeded. This is the path used for uploads.

2. `Database.ensure_schema(seed=True)`: - creates parent directories - executes DDL for `departments` and `employees` - inserts `SEED_DEPARTMENTS` and `SEED_EMPLOYEES` using `INSERT OR IGNORE` - guarded by `_init_lock` and `_initialized` flag for idempotency

Tables and seed columns:

1. `departments(dept_id, dept_name, location)`
2. `employees(emp_id, name, salary, dept_id)` with FK `employees.dept_id -> departments.dept_id`

3.6 Flow E: Safety gate behavior details

`prepare_sql()` in `security/sql_validator.py` enforces:

1. SQL must parse and be non-empty (`parse_sql`).
2. Exactly one statement.
3. Top-level must be `SELECT` or set op (`UNION/INTERSECT/EXCEPT`) if policy allows.

4. Optional policy bans: - subqueries - joins - aggregates - CTEs - union
5. Dangerous functions are blocked (`load_extension`, `readfile`, `writefile`, `edit`, `shell`, `system`).
6. Additional banned keyword scan blocks destructive operations.
7. JOIN, subquery, and CTE counts stay within configured limits.
8. Every referenced physical table belongs to the current allowlist. CTE names are derived sources, but same-name physical tables are still authorized independently through scope traversal.
9. Preparation reuses one parsed AST for validation, table discovery, and LIMIT canonicalization; a regression test enforces the single-parse path.
10. The returned SQL contains an enforced result LIMIT.

After AST checks, `Database.preflight()` validates identifiers without running the SELECT. Execution uses SQLite URI `mode=ro, query_only`, disabled extensions, `trusted_schema=OFF`, an elapsed-time deadline, and a VM-step cap.

Output contract: - Success: returns `PreparedSQL(sql=..., tables=...)`. - Failure: raises `SQLValidationError` with user-facing message.

Why this matters for learners:

1. This safety gate is the key trust boundary in the system.
2. It runs before query execution.
3. It is configurable through `Settings` flags that map to `SQLPolicy`.

Module 4: Setup & Run Guide

This section is static and based on repo files (`README.md`, `pyproject.toml`, source).

4.1 Prerequisites

1. Python `3.12.10` (from `pyproject.toml`: `requires-python = ">=3.12.10,<3.13"`).
2. `uv` for environment/dependency management.
3. Optional but default runtime path: local Ollama (`NL2SQL_PROVIDER=ollama`).

4.2 Install on a clean machine

```
git clone <repo-url> cd natural-language-to-sql-agent uv venv --python 3.12.10 uv sync --all-groups uv pip install -e .
```

4.3 Environment configuration (`.env` or shell env)

Core `Settings` keys (`src/nl2sql_agent/config/settings.py`)

Use these as `NL2SQL_*` variables:

1. `NL2SQL_PROVIDER`
2. `NL2SQL_MODEL`
3. `NL2SQL_OLLAMA_BASE_URL`
4. `NL2SQL_OLLAMA_KEEP_ALIVE`
5. `NL2SQL_DB_PATH`
6. `NL2SQL_DB_SEED`

7. NL2SQL_DB_MAX_ROWS
8. NL2SQL_DB_QUERY_TIMEOUT_SECONDS
9. NL2SQL_DB_MAX_VM_STEPS
10. NL2SQL_DB_UPLOAD_MAX_MB
11. NL2SQL_MAX_RETRIES
12. NL2SQL_LLM_TEMPERATURE
13. NL2SQL_LLM_MAX_TOKENS
14. NL2SQL_LLM_REQUEST_TIMEOUT_SECONDS
15. NL2SQL_SQL_ALLOW_SUBQUERIES
16. NL2SQL_SQL_ALLOW_JOINS
17. NL2SQL_SQL_ALLOW_AGGREGATES
18. NL2SQL_SQL_ALLOW_CTE
19. NL2SQL_SQL_MAX_JOINS
20. NL2SQL_SQL_MAX_SUBQUERIES
21. NL2SQL_SQL_MAX_CTES
22. NL2SQL_SCHEMA_MAX_TABLES
23. NL2SQL_AUDIT_ENABLED
24. NL2SQL_AUDIT_PATH
25. NL2SQL_LOG_LEVEL
26. NL2SQL_LOG_JSON

Provider API key env names used by UI helper

env_var_for() in settings.py maps:

1. openai -> OPENAI_API_KEY
2. gemini -> GOOGLE_API_KEY
3. anthropic -> ANTHROPIC_API_KEY
4. huggingface -> HF_TOKEN
5. xai -> XAI_API_KEY

Minimal .env examples:

```
# Local-first default path NL2SQL_PROVIDER=ollama NL2SQL_MODEL=phi4-mini:3.8b
NL2SQL_OLLAMA_BASE_URL=http://localhost:11434 NL2SQL_DB_PATH=company.db NL2SQL_DB_SEED=true
```

```
# Cloud provider example (OpenAI) NL2SQL_PROVIDER=openai NL2SQL_MODEL=gpt-5.6-luna
OPENAI_API_KEY=your key here NL2SQL_DB_PATH=company.db
```

Hosted requests use medium reasoning. OpenAI allows only `gpt-5.6-luna` and `gpt-5.6-terra`; Anthropic allows `claude-sonnet-5`; Gemini allows `gemini-3.7-flash` and `gemini-3.5-flash-lite`; xAI allows `grok-4.6`. Hugging Face accepts `namespace/model[:routing-policy]` through its direct router and defaults to `openai/gpt-oss-120b:fastest`.

4.4 Typical run commands

Start Streamlit UI

On Windows, double-click `Launch NL2SQL Agent.cmd`. The terminal equivalent is:

```
uv run streamlit run src/nl2sql_agent/ui/streamlit_app.py
```

Ask one question in CLI

```
uv run nl2sql-agent ask "How many employees are in each department?" uv run nl2sql-agent ask --show-sql "What is the total salary in Engineering?"
```

Show resolved runtime config

```
uv run nl2sql-agent config
```

Serve command wrapper (invokes Streamlit via CLI)

```
uv run nl2sql-agent serve --host 127.0.0.1 --port 8501
```

Run the result and safety corpus

```
uv run nl2sql-agent eval --min-pass-rate 0.4
```

4.5 Database migration/seeding steps

There is no migration framework (no Alembic/Flyway-style migration scripts in this repo).

Current behavior:

1. Schema creation is runtime and idempotent via `Database.ensure_schema()`.
2. Seeding is controlled by `db_seed` (`NL2SQL_DB_SEED`, default `True`).
3. Backward-compatible helper exists: `setup_db(path, seed=True)`.

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan (zero to hero)

1. Read `README.md` sections: architecture, workflow, configuration, CLI/UI.
2. Read `src/nl2sql_agent/config/settings.py` to understand runtime knobs.
3. Read `src/nl2sql_agent/db/database.py` + `db/seed.py` for actual data model.
4. Read `src/nl2sql_agent/security/sql_validator.py` for guardrail logic.
5. Read `src/nl2sql_agent/prompts/templates.py` for prompt contracts.
6. Read `src/nl2sql_agent/llm/factory.py` for provider abstraction.
7. Read `src/nl2sql_agent/agent/state.py` then `agent/workflow.py` linearly.
8. Read `src/nl2sql_agent/ui/streamlit_app.py` + `ui/components.py`.
9. Read `src/nl2sql_agent/cli.py`.
10. Confirm understanding through tests:
 - `tests/unit/test_agent.py`
 - `tests/unit/test_sql_validator.py`
 - `tests/unit/test_database.py`

- `tests/unit/test_cli.py`

5.2 Practice exercises

Trace one full run: Starting from `cmd_ask`, list each function called until `final_answer` is printed. File focus: `src/nl2sql_agent/cli.py`, `src/nl2sql_agent/agent/workflow.py`.

State-contract drill: Write the minimal `AgentState` keys required for `write_sql()` to run without missing context. File focus: `src/nl2sql_agent/agent/state.py`, `src/nl2sql_agent/agent/workflow.py`.

Security policy scenario: Explain what happens if `allow_joins=False` and the model generates a JOIN query. File focus: `src/nl2sql_agent/security/sql_validator.py`.

Retry logic reasoning: For `retry_count=2`, `max_retries=3`, and execution error present, what node runs next and why? File focus: `src/nl2sql_agent/agent/workflow.py`.

Data contract check: What are the exact types for `columns`, `raw_rows`, and `row_count` returned by `execute_sql()`? File focus: `src/nl2sql_agent/db/database.py`, `src/nl2sql_agent/agent/workflow.py`.

Config mapping task: Map one settings field to an environment variable and describe where it affects runtime behavior. File focus: `src/nl2sql_agent/config/settings.py`.

UI behavior task: Explain why live `list_models()` discovery is Ollama-only and how hosted model choices are supplied. File focus: `src/nl2sql_agent/ui/streamlit_app.py`, `src/nl2sql_agent/ui/components.py`.

Extension design: Trace the implemented xAI path from settings and credentials through the factory and sidebar. File focus: `src/nl2sql_agent/config/settings.py`, `src/nl2sql_agent/llm/factory.py`, `src/nl2sql_agent/ui/components.py`.

Approval flow: Explain why editing SQL in Streamlit cannot bypass the guardian. File focus: `src/nl2sql_agent/ui/streamlit_app.py`, `src/nl2sql_agent/agent/workflow.py`.

Evaluation scoring: Explain why reference-result comparison accepts multiple correct SQL forms. File focus: `src/nl2sql_agent/evaluation/runner.py`, `src/nl2sql_agent/evaluation/data/demo.jsonl`.

5.3 Solution outlines

Exercise 1 outline: `main -> parse_args -> cmd_ask -> _build_agent -> get_settings -> build_chat_model -> NL2SQLAgent.run -> get_workflow/invoke -> final dict -> print(final_answer)`.

Exercise 2 outline: Minimum practical keys are `question`, `schema`; optional prior context is `error` or `sql_unsafe_reason`; `retry_count` defaults to 0 if missing.

Exercise 3 outline: `check_security` calls `prepare_sql`; the policy raises `SQLValidationError("JOIN clauses are not allowed.")`; the node returns `error + sql_unsafe_reason`; `route_after_security` retries `writer` while budget remains, then ends at `summarizer`.

Exercise 4 outline: `route_after_execute` checks `err` and `retry < max_retries`; with `2 < 3`, it routes to "writer" for another SQL rewrite attempt.

Exercise 5 outline: `execute_sql` returns `columns: list[str]`, `raw_rows: list[tuple[object, ...]]`, `row_count: int`.

Exercise 6 outline: Example: `Settings.db_max_rows` from `NL2SQL_DB_MAX_ROWS` influences `Database(max_rows=...)`; `execute()` fetches `max_rows + 1` then truncates.

Exercise 7 outline: `list_models()` performs a network call only after `refresh_trigger_ollama` is set. Hosted providers read deterministic choices from the central settings catalog; Hugging Face additionally accepts a custom repository ID.

Exercise 8 outline: `Provider` includes `xai`; `env_var_for()` maps `XAI_API_KEY`; `_build_xai()` targets the fixed HTTPS endpoint with medium reasoning; the sidebar exposes only `grok-4.6`.

Exercise 9 outline: `execute_prepared()` replaces the candidate with the edited SQL, calls `check_security()` again, and only reaches `execute_sql()` when the second validation and preflight succeed.

Exercise 10 outline: The evaluator executes curated reference SQL read-only and compares result values. SQL aliases, formatting, and equivalent query structure do not have to match; ordered cases still require the requested row order.

Exercise 11 outline: `write_sql()` and `summarize()` retain per-call provider usage in `AgentState.usage_records`; `calculate_cost()` selects the effective rule and actual mode for each call, including cache and long-context adjustments. Approved runs freeze the price snapshot. Unknown Ollama or custom Hugging Face models remain unpriced until a matching rule is configured.

Verification Checklist

Use this to self-check mastery:

1. Can you explain the full node order in `NL2SQLAgent` and both routing decisions?
2. Can you describe exactly how unsafe SQL is blocked before execution?
3. Can you enumerate the most important `AgentState` keys and what writes each one?
4. Can you explain how retries are triggered and when they stop?
5. Can you show where schema creation and seed insertion occur?
6. Can you explain how provider/model selection flows from UI/CLI into `build_chat_model()`?
7. Can you point to where final answer fallback behavior is implemented?
8. Can you map at least five environment settings to their runtime effects?
9. Can you explain how uploaded databases stay session-scoped and read-only?
10. Can you interpret the accuracy, safety, latency, retry, and token fields in an evaluation report?
11. Can you explain the UI cost formula and why conditional provider discounts are not applied to aggregate run usage?